3 METHOD

In this section, we first introduce the concept of the null space and its relationship to model editing (Section 3.1). Based on this, we present the method for projecting a given perturbation $\boldsymbol{\Delta}$ onto the null space of the matrix $\boldsymbol{K}_0$, which encodes the persevered knowledge (Section 3.2). Following that, a new editing objective that incorporates the aforementioned projection method is introduced in Section 3.3.

3.1 NULL SPACE

Null space is at the core of our work. Here we first introduce the definition of the left null space (hereafter referred to simply as *null space*). Specifically, given two matrices $\boldsymbol{A}$ and $\boldsymbol{B}$, $\boldsymbol{B}$ is in the null space of $\boldsymbol{A}$ if and only if $\boldsymbol{BA} = \boldsymbol{0}$. See Adam-NSCL (Wang et al., 2021) for more details.

In the context of model editing, if the perturbation $\boldsymbol{\Delta}$ is projected into the null space of $\boldsymbol{K}_0$ (*i.e.*, $\boldsymbol{\Delta}'\boldsymbol{K}_0 = \boldsymbol{0}$, where $\boldsymbol{\Delta}'$ denotes the projected perturbation), adding it to the parameters $\boldsymbol{W}$ results in:

$$(\boldsymbol{W} + \boldsymbol{\Delta}')\boldsymbol{K}_0 = \boldsymbol{W}\boldsymbol{K}_0 = \boldsymbol{V}_0. \tag{7}$$

This implies that **the projected $\boldsymbol{\Delta}$ will not disrupt the key-value associations of the preserved knowledge** (*i.e.*, $\{\boldsymbol{K}_0, \boldsymbol{V}_0\}$), ensuring that the storage of the preserved knowledge remains intact.

Therefore, in this work, before adding perturbation $\boldsymbol{\Delta}$ to the model parameters $\boldsymbol{W}$, we project it onto the null space of $\boldsymbol{K}_0$ to protect the preserved knowledge. This protection allows us to remove the first term — the term focusing on protecting the preserved knowledge — from the objective in Eqn. 5.

3.2 NULL SPACE PROJECTING

In Section 3.1, we briefly explained why $\boldsymbol{\Delta}$ should be projected into the null space of $\boldsymbol{K}_0$. In this part, we focus on how to implement this projection.

As introduced at the end of Section 2.2, the matrix $\boldsymbol{K}_0 \in \mathbb{R}^{d_0 \times 100{,}000}$ has a high dimensionality with $100,000$ columns. Hence, directly projecting the given perturbation $\boldsymbol{\Delta}$ onto the null space of $\boldsymbol{K}_0$ presents significant computational and storage challenges. In sight of this, we adopt the null space of the non-central covariance matrix $\boldsymbol{K}_0\boldsymbol{K}_0^T \in \mathbb{R}^{d_0 \times d_0}$ as a substitute to reduce computational complexity, as d_0 is typically much smaller than $100,000$. This matrix's null space is equal to that of $\boldsymbol{K}_0$ (please see Appendix B.2 for detailed proof).

Following the existing methods for conducting null space projection (Wang et al., 2021), we first apply a Singular Value Decomposition (SVD) to $\boldsymbol{K}_0(\boldsymbol{K}_0)^T$:

$$\left\{\boldsymbol{U}, \boldsymbol{\Lambda}, (\boldsymbol{U})^T\right\} = \text{SVD}\left(\boldsymbol{K}_0(\boldsymbol{K}_0)^T\right), \tag{8}$$

where each column in $\boldsymbol{U}$ is an eigenvector of $\boldsymbol{K}_0(\boldsymbol{K}_0)^T$. Then, we remove the eigenvectors in $\boldsymbol{U}$ that correspond to non-zero eigenvalues[1], and define the remaining submatrix as $\hat{\boldsymbol{U}}$. Based on this, the projection matrix $\boldsymbol{P}$ can be defined as follows:

$$\boldsymbol{P} = \hat{\boldsymbol{U}}(\hat{\boldsymbol{U}})^T. \tag{9}$$

This projection matrix can map the column vectors of $\boldsymbol{\Delta}$ into the null space of $\boldsymbol{K}_0(\boldsymbol{K}_0)^T$, as it satisfies the condition $\boldsymbol{\Delta P} \cdot \boldsymbol{K}_0(\boldsymbol{K}_0)^T = \boldsymbol{0}$. The detailed derivation is exhibited in Appendix B.3.

Since $\boldsymbol{K}_0$ and $\boldsymbol{K}_0(\boldsymbol{K}_0)^T$ share the same null space, we can derive $\boldsymbol{\Delta P} \cdot \boldsymbol{K}_0 = \boldsymbol{0}$. Hence, we have:

$$(\boldsymbol{W} + \boldsymbol{\Delta P})\boldsymbol{K}_0 = \boldsymbol{W}\boldsymbol{K}_0 = \boldsymbol{V}_0. \tag{10}$$

This shows the projection matrix $\boldsymbol{P}$ ensures that the model edits occur without interference with the preserved knowledge in LLMs.

3.3 NULL-SPACE CONSTRAINED MODEL EDITING

Section 3.2 has provided how to apply projection to ensure that preserved knowledge is not disrupted. Here, we introduce how to leverage this projection to optimize the current model editing objective.

[1]Given that eigenvalues are rarely strictly zero in practical applications, in our experiments, we remove the eigenvectors corresponding to the eigenvalues above 10^{-2}.

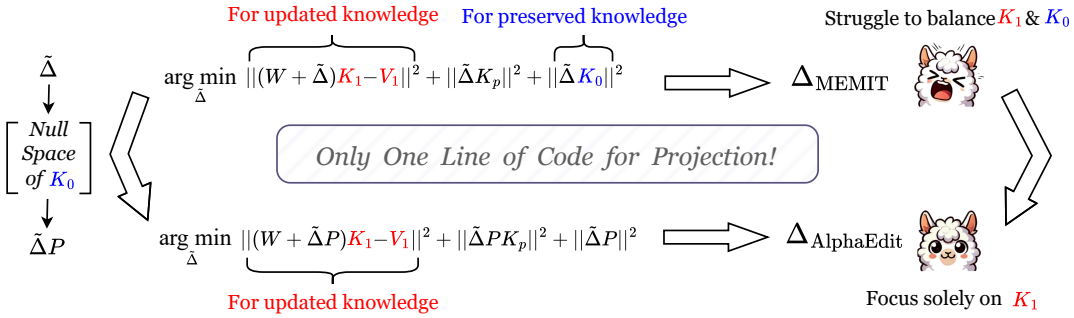


Figure 3: Comparison between the paradigms of AlphaEdit and current method. Best viewed in color.

Starting with the single-edit objective in Eqn. 5, the optimization follows three steps: (1) Replace $\boldsymbol{\Delta}$ with the projected perturbation $\boldsymbol{\Delta P}$, ensuring that the perturbation would not disrupt the preserved knowledge; (2) Remove the first term involving $\boldsymbol{K}_0$, as Step (1) has already protected the preserved knowledge from being disrupted; (3) Add a regularization term $||\boldsymbol{\Delta P}||^2$ to guarantee stable convergence. With these optimizations, Eqn. 5 becomes:

$$\boldsymbol{\Delta} = \arg\min_{\tilde{\boldsymbol{\Delta}}} \left(||(\boldsymbol{W} + \tilde{\boldsymbol{\Delta}}\boldsymbol{P})\boldsymbol{K}_1 - \boldsymbol{V}_1||^2 + \|\tilde{\boldsymbol{\Delta}}\boldsymbol{P}\|^2 \right), \tag{11}$$

where $\boldsymbol{K}_1$ and $\boldsymbol{V}_1$ denote the key and value matrices of to-be-updated knowledge defined in Eqn. 3.

In sequential editing tasks, during the current edit, we also need to add a term to the objective (*cf.* Eqn. 11) to prevent the perturbation from disrupting the updated knowledge in previous edits. Let $\boldsymbol{K}_p$ and $\boldsymbol{V}_p$ present the key and value matrices of the previously updated knowledge, analogous to the earlier definitions of $\boldsymbol{K}_1$ and $\boldsymbol{V}_1$. This term should minimize $||(\boldsymbol{W} + \tilde{\boldsymbol{\Delta}}\boldsymbol{P})\boldsymbol{K}_p - \boldsymbol{V}_p||^2$ to protect the association. Since the related knowledge has been updated in previous edits, we have $\boldsymbol{W}\boldsymbol{K}_p = \boldsymbol{V}_p$. Hence, this term can be simplified to $||\tilde{\boldsymbol{\Delta}}\boldsymbol{P}\boldsymbol{K}_p||^2$, and adding it to Eqn. 11 gives the new objective:

$$\boldsymbol{\Delta} = \arg\min_{\tilde{\boldsymbol{\Delta}}} \left(||(\boldsymbol{W} + \tilde{\boldsymbol{\Delta}}\boldsymbol{P})\boldsymbol{K}_1 - \boldsymbol{V}_1||^2 + \|\tilde{\boldsymbol{\Delta}}\boldsymbol{P}\|^2 + ||\tilde{\boldsymbol{\Delta}}\boldsymbol{P}\boldsymbol{K}_p||^2 \right). \tag{12}$$

To facilitate expression, we define the residual vector of the current edit as $\boldsymbol{R} = \boldsymbol{V}_1 - \boldsymbol{W}\boldsymbol{K}_1$. Based on this, Eqn. 12 can be solved using the normal equation (Lang, 2012):

$$(\boldsymbol{\Delta}\boldsymbol{P}\boldsymbol{K}_1 - \boldsymbol{R})\boldsymbol{K}_1^T\boldsymbol{P} + \boldsymbol{\Delta}\boldsymbol{P} + \boldsymbol{\Delta}\boldsymbol{P}\boldsymbol{K}_p\boldsymbol{K}_p^T\boldsymbol{P} = \boldsymbol{0}. \tag{13}$$

Solving Eqn. 13 yields the final perturbation $\boldsymbol{\Delta}_{\textbf{AlphaEdit}} = \boldsymbol{\Delta P}$ which will be added to the model parameters $\boldsymbol{W}$:

$$\boldsymbol{\Delta}_{\textbf{AlphaEdit}} = \boldsymbol{R}\boldsymbol{K}_1^T\boldsymbol{P}(\boldsymbol{K}_p\boldsymbol{K}_p^T\boldsymbol{P} + \boldsymbol{K}_1\boldsymbol{K}_1^T\boldsymbol{P} + \boldsymbol{I})^{-1}. \tag{14}$$

The detailed derivation process and the invertibility of the term within the brackets are provided in Appendices B.4 and B.5 respectively. This solution $\boldsymbol{\Delta}_{\textbf{AlphaEdit}}$ could not only store the to-be-updated knowledge in the current edit, but also ensure that both the preserved knowledge and the previously updated knowledge remain unaffected. Furthermore, for better comparison, we also present the commonly used solution in existing methods like MEMIT (Meng et al., 2023) as follows[2]:

$$\boldsymbol{\Delta}_{\textbf{MEMIT}} = \boldsymbol{R}\boldsymbol{K}_1^T(\boldsymbol{K}_p\boldsymbol{K}_p^T + \boldsymbol{K}_1\boldsymbol{K}_1^T + \boldsymbol{K}_0\boldsymbol{K}_0^T)^{-1}. \tag{15}$$

By comparing Eqn. 14 and 15, it is evident that our approach requires only a minor modification to the standard solution by incorporating the projection matrix $\boldsymbol{P}$. This makes our method easily integrable into existing model editing algorithms. Figure 3 summarizes this modification from the perspective of convergence objectives. We emphasize that **by adding just a single line of code for this modification, the performance of most editing methods could be significantly enhanced**, as shown in Figure 2. More detailed experimental results are exhibited in Section 4.

Furthermore, since the projection matrix $\boldsymbol{P}$ is entirely independent of the to-be-updated knowledge, it only needs to be computed once and can then be directly applied to any downstream editing tasks. Consequently, AlphaEdit introduces negligible additional time consumption compared to baselines, making it both efficient and effective.

[2] $\Delta_{\textbf{MEMIT}}$ denotes the solution provided by MEMIT in sequential editing tasks.

Table 1: Comparison of AlphaEdit with existing methods on the sequential model editing task. *Eff.*, *Gen.*, *Spe.*, *Flu.* and *Consis.* denote Efficacy, Generalization, Specificity, Fluency and Consistency, respectively. The best results are highlighted in bold, while the second-best results are underlined.

Method	Model	Counterfact					ZsRE		
		Eff.↑	Gen.↑	Spe.↑	Flu.↑	Consis.↑	Eff.↑	Gen.↑	Spe.↑
Pre-edited		7.85±0.26	10.58±0.26	89.48±0.18	635.23±0.11	24.14±0.08	36.99±0.30	36.34±0.30	31.89±0.22
FT	LLaMA3	83.33±0.37	67.79±0.40	46.63±0.37	233.72±0.22	8.77±0.05	30.48±0.26	30.22±0.32	15.49±0.17
MEND	LLaMA3	63.24±0.31	61.17±0.36	45.37±0.38	372.16±0.80	4.21±0.05	0.91±0.05	1.09±0.05	0.53±0.02
InstructEdit	LLaMA3	66.58±0.24	64.18±0.35	47.14±0.37	443.85±0.78	7.28±0.04	1.58±0.04	1.36±0.08	1.01±0.05
ROME	LLaMA3	64.40±0.41	61.42±0.42	49.44±0.38	449.06±0.26	3.31±0.02	2.01±0.07	1.80±0.07	0.69±0.03
MEMIT	LLaMA3	65.65±0.47	64.65±0.42	51.56±0.38	437.43±1.67	6.58±0.11	34.62±0.36	31.28±0.34	18.49±0.19
PRUNE	LLaMA3	68.25±0.46	64.75±0.41	49.82±0.36	418.03±1.52	5.90±0.10	24.77±0.27	23.87±0.27	20.69±0.23
RECT	LLaMA3	66.05±0.47	63.62±0.43	61.41±0.37	526.62±0.44	20.54±0.09	86.05±0.23	80.54±0.27	31.67±0.22
AlphaEdit	LLaMA3	**98.90±0.10**	**94.22±0.19**	**67.88±0.29**	**622.49±0.16**	**32.40±0.11**	**94.47±0.13**	**91.13±0.19**	**32.55±0.22**
Pre-edited		16.22±0.31	18.56±0.45	83.11±0.13	621.81±0.67	29.74±0.51	26.32±037	25.79±0.25	27.42±0.53
FT	GPT-J	92.15±0.27	72.38±0.38	43.35±0.37	297.92±0.77	6.65±0.10	72.37±0.29	68.91±0.32	19.66±0.23
MEND	GPT-J	46.15±0.50	46.22±0.51	53.90±0.48	242.41±0.41	3.94±0.03	0.71±0.04	0.71±0.04	0.52±0.03
InstructEdit	GPT-J	50.62±0.58	51.73±0.42	56.28±0.50	245.89±0.44	4.21±0.04	0.92±0.07	0.88±0.03	0.65±0.06
ROME	GPT-J	57.50±0.48	54.20±0.40	52.05±0.31	589.42±0.08	3.22±0.02	56.42±0.42	54.65±0.42	9.86±0.16
MEMIT	GPT-J	98.55±0.11	95.50±0.16	63.64±0.31	546.28±0.88	34.89±0.15	94.91±0.16	90.22±0.23	**30.39±0.27**
PRUNE	GPT-J	86.15±0.34	86.85±0.29	53.87±0.35	427.14±0.53	14.78±0.11	0.15±0.02	0.15±0.02	0.00±0.00
RECT	GPT-J	98.80±0.10	86.58±0.28	72.22±0.28	617.31±0.19	41.39±0.12	96.38±0.14	91.21±0.21	27.79±0.26
AlphaEdit	GPT-J	**99.75±0.08**	**96.38±0.23**	**75.48±0.21**	**618.50±0.17**	**42.08±0.15**	**99.79±0.14**	**96.00±0.22**	28.29±0.25
Pre-edited		22.23±0.73	24.34±0.62	78.53±0.33	626.64±0.31	31.88±0.20	22.19±0.24	31.30±0.27	24.15±0.32
FT	GPT2-XL	63.55±0.48	42.20±0.41	57.06±0.30	519.35±0.27	10.56±0.05	37.11±0.39	33.30±0.37	10.36±0.17
MEND	GPT2-XL	50.80±0.50	50.80±0.48	49.20±0.51	407.21±0.08	1.01±0.00	0.00±0.00	0.00±0.00	0.00±0.00
InstructEdit	GPT2-XL	55.32±0.58	53.63±0.42	53.25±0.62	412.57±0.15	1.08±0.03	3.54±0.03	4.25±0.02	3.23±0.04
ROME	GPT2-XL	54.60±0.48	51.18±0.40	52.68±0.33	366.13±1.40	0.72±0.02	47.50±0.43	43.56±0.42	14.27±0.19
MEMIT	GPT2-XL	94.70±0.22	85.82±0.28	60.50±0.32	477.26±0.54	22.72±0.15	79.17±0.32	71.44±0.36	**26.42±0.25**
PRUNE	GPT2-XL	82.05±0.38	78.55±0.34	53.02±0.35	530.47±0.39	15.93±0.11	21.62±0.30	19.27±0.28	13.19±0.18
RECT	GPT2-XL	92.15±0.26	81.15±0.33	65.13±0.31	480.83±0.62	21.05±0.16	81.02±0.31	73.08±0.35	24.85±0.25
AlphaEdit	GPT2-XL	**99.50±0.24**	**93.95±0.34**	**66.39±0.31**	**597.88±0.18**	**39.38±0.15**	**94.81±0.30**	**86.11±0.29**	25.88±0.21

4 EXPERIMENT

In this section, we conduct experiments to address the following research questions:

- **RQ1:** How does AlphaEdit perform on sequential editing tasks compared to baseline methods? Can it mitigate the issues of model forgetting and model collapse exhibited in Figure 1?
- **RQ2:** How does AlphaEdit-edited LLM perform on general ability evaluations? Does the post-edited LLM successfully retain its inherent capabilities?
- **RQ3:** Can AlphaEdit effectively prevent the model from overfitting to updated knowledge? Specifically, can the post-edited LLM avoid shifts in the distribution of hidden representations?
- **RQ4:** Can the performance of baseline methods be significantly improved with a single line of code in AlphaEdit (*i.e.*, the code for projection)?

4.1 EXPERIMENTAL SETUP

We begin by briefly outlining the evaluation metrics, datasets, and baseline methods. For more detailed descriptions of the experimental settings, please refer to Appendix A.

Base LLMs & Baseline Methods. Our experiments are conducted on three LLMs: GPT2-XL (1.5B), GPT-J (6B), and LLaMA3 (8B). We compare our method against several model editing baselines, including Fine-Tuning (FT) (Zhu et al., 2020), MEND (Mitchell et al., 2022a), InstructEdit (Zhang et al., 2024b), ROME (Meng et al., 2022), MEMIT (Meng et al., 2023), PRUNE (Ma et al., 2024), and